Projet final Java + UML

Application de gestion d'une bibliothèque universitaire

1. Contexte du projet

Dans le cadre du cours **Java + UML**, il est demandé de réaliser un projet final permettant de mettre en pratique les notions étudiées pendant le cours.

Le projet consiste à développer une application Java de gestion d'une bibliothèque universitaire.
L'application doit permettre de gérer les documents, les utilisateurs, les emprunts et les retours de documents.

Le projet doit être réalisé en utilisant les principes de la **programmation orientée objet**, accompagné d'une modélisation **UML**, et organisé proprement en **packages Java** sous IntelliJ IDEA ou Eclipse.

---

2. Objectifs pédagogiques

Ce projet a pour objectif d'évaluer la capacité de l'étudiant à :

- modéliser une application avec UML ;
- créer une application Java structurée ;
- utiliser les classes, objets, attributs, méthodes et constructeurs ;
- appliquer l'encapsulation ;
- utiliser l'héritage et le polymorphisme ;
- utiliser les classes génériques ;
- utiliser les collections Java ;
- gérer les erreurs à l'aide des exceptions ;
- organiser un projet Java en packages ;
- développer une application fonctionnelle avec une interface console.

---

3. Description générale de l'application

L'application doit permettre la gestion d'une bibliothèque universitaire.

Elle doit permettre de gérer :

- les documents disponibles dans la bibliothèque ;
- les utilisateurs de la bibliothèque ;

- les emprunts de documents ;
- les retours de documents ;
- la recherche de documents ;
- l'affichage des documents et des emprunts.

L'application sera développée en mode **console**.

---

4. Gestion des documents

La bibliothèque contient plusieurs types de documents :

- Livre ;
- Magazine ;
- Mémoire universitaire.

Tous les documents doivent avoir au minimum les attributs suivants :

- identifiant ;
- titre ;
- auteur ;
- année de publication ;
- statut : disponible ou emprunté.

Chaque type de document doit avoir des attributs spécifiques.

Exemple pour la classe Livre

Un livre peut avoir :

- nombre de pages ;
- genre.

Exemple pour la classe Magazine

Un magazine peut avoir :

- numéro d'édition ;
- mois de publication.

Exemple pour la classe Memoire

Un mémoire peut avoir :

- niveau : Licence, Master ou Doctorat ;
- encadrant.

Il faut créer une classe mère `Document`, puis des classes filles comme `Livre`, `Magazine` et `Memoire`.

Les classes filles doivent hériter de la classe `Document`.

---

5. Gestion des utilisateurs

La bibliothèque contient plusieurs types d'utilisateurs :

- Étudiant ;
- Enseignant.

Tous les utilisateurs doivent avoir au minimum :

- identifiant ;
- nom ;
- email.

Chaque type d'utilisateur doit avoir des attributs spécifiques.

Exemple pour la classe Etudiant

Un étudiant peut avoir :

- filière ;
- niveau.

Exemple pour la classe Enseignant

Un enseignant peut avoir :

- département ;
- spécialité.

Il faut créer une classe mère `Utilisateur`, puis des classes filles `Etudiant` et `Enseignant`.

Les classes filles doivent hériter de la classe `Utilisateur`.

---

6. Gestion des emprunts

L'application doit permettre à un utilisateur d'emprunter un document disponible.

Un emprunt doit contenir au minimum :

- un utilisateur ;
- un document ;

- une date d’emprunt ;
- une date de retour prévue.

L’application doit permettre de :

- emprunter un document ;
- retourner un document ;
- afficher les documents empruntés par un utilisateur ;
- afficher tous les emprunts en cours.

Un document déjà emprunté ne peut pas être emprunté une deuxième fois avant son retour.

---

7. Fonctionnalités obligatoires

L’application doit contenir au minimum les fonctionnalités suivantes :

1. Ajouter un document ;
2. Ajouter un utilisateur ;
3. Afficher tous les documents ;
4. Afficher tous les utilisateurs ;
5. Rechercher un document par son identifiant ou son titre ;
6. Emprunter un document ;
7. Retourner un document ;
8. Afficher les documents empruntés par un utilisateur ;
9. Afficher tous les emprunts en cours ;
10. Quitter l’application.

---

8. Interface console attendue

L’application doit proposer un menu console simple.

Exemple :

```
===== Gestion Bibliothèque =====
1. Ajouter un document
2. Ajouter un utilisateur
3. Afficher tous les documents
4. Afficher tous les utilisateurs
5. Rechercher un document
6. Emprunter un document
7. Retourner un document
8. Afficher les emprunts d’un utilisateur
```

```
9. Afficher tous les emprunts
10. Quitter
Votre choix :
```

L'interface graphique n'est pas demandée.
L'objectif principal est d'avoir une application Java bien structurée, claire et fonctionnelle.

---

9. Utilisation obligatoire des collections Java

Le projet doit utiliser les collections Java.

Exemples de collections possibles :

```
ArrayList<Document> documents;
ArrayList<Utilisateur> utilisateurs;
HashMap<Utilisateur, ArrayList<Document>> emprunts;
HashSet<String> emailsUtilisateurs;
```

Utilisation attendue :

- `ArrayList` pour stocker les documents ;
- `ArrayList` pour stocker les utilisateurs ;
- `HashMap` pour associer un utilisateur à ses documents empruntés ;
- `HashSet` pour éviter les doublons, par exemple les emails ou les identifiants.

L'utilisation des collections doit être justifiée dans le rapport.

---

10. Utilisation obligatoire des exceptions

Le projet doit contenir des exceptions personnalisées.

Exemples d'exceptions possibles :

```
DocumentIndisponibleException
DocumentIntrouvableException
UtilisateurIntrouvableException
EmpruntImpossibleException
EmailDejaExisteException
```

Cas d'erreurs à gérer :

- emprunter un document inexistant ;
- emprunter un document déjà emprunté ;
- retourner un document qui n'est pas emprunté ;
- ajouter un utilisateur avec un email déjà existant ;

- rechercher un document inexistant ;
- saisir un choix invalide dans le menu.

Les exceptions doivent être utilisées avec `try`, `catch`, `throw` et éventuellement `throws`.

---

11. Utilisation obligatoire d'une classe générique

Le projet doit contenir au moins une classe générique.

Exemple :

```java
public class Gestionnaire<T> {
    private ArrayList<T> elements;

    public Gestionnaire() {
        elements = new ArrayList<>();
    }

    public void ajouter(T element) {
        elements.add(element);
    }

    public void supprimer(T element) {
        elements.remove(element);
    }

    public ArrayList<T> getElements() {
        return elements;
    }
}
```

Cette classe peut être utilisée pour gérer plusieurs types d'objets :

```java
Gestionnaire<Document> gestionDocuments;
Gestionnaire<Utilisateur> gestionUtilisateurs;
```

L'objectif est de montrer la compréhension du principe des classes génériques en Java.

---

12. Organisation obligatoire en packages

Le projet doit être organisé en packages Java.

Exemple d'organisation possible :

```
src/
 └── ma.ensate.bibliotheque/
      ├── Main.java
```

```
│
├── models/
│     ├── Document.java
│     ├── Livre.java
│     ├── Magazine.java
│     ├── Memoire.java
│     ├── Utilisateur.java
│     ├── Etudiant.java
│     ├── Enseignant.java
│     └── Emprunt.java
│
├── services/
│     ├── BibliothequeService.java
│     └── Gestionnaire.java
│
├── exceptions/
│     ├── DocumentIndisponibleException.java
│     ├── DocumentIntrouvableException.java
│     ├── UtilisateurIntrouvableException.java
│     ├── EmpruntImpossibleException.java
│     └── EmailDejaExisteException.java
│
└── app/
      └── MenuConsole.java
```

Chaque classe doit être placée dans le package correspondant.

Exemple :

```
package ma.ensate.bibliotheque.models;
```

ou :

```
package ma.ensate.bibliotheque.exceptions;
```

Le projet peut être réalisé avec **IntelliJ IDEA** ou **Eclipse**.

---

13. UML demandé

Le projet doit être accompagné d'une modélisation UML.

13.1 Diagramme de classes

Le diagramme de classes doit contenir au minimum :

- `Document`
- `Livre`
- `Magazine`

- `Memoire`
- `Utilisateur`
- `Etudiant`
- `Enseignant`
- `Emprunt`
- `BibliothequeService`
- `Gestionnaire<T>`
- les exceptions personnalisées

Le diagramme doit montrer :

- les attributs ;
- les méthodes ;
- les relations d'héritage ;
- les associations ;
- les multiplicités si possible.

13.2 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation doit contenir au minimum les acteurs suivants :

- Bibliothécaire ;
- Utilisateur.

Cas d'utilisation possibles :

- ajouter un document ;
- ajouter un utilisateur ;
- rechercher un document ;
- emprunter un document ;
- retourner un document ;
- afficher les emprunts.

---

14. Contraintes techniques

Le projet doit respecter les contraintes suivantes :

- utiliser le langage Java ;
- utiliser la programmation orientée objet ;
- utiliser l'encapsulation ;
- utiliser l'héritage ;
- utiliser le polymorphisme ;

- utiliser au moins une classe générique ;
- utiliser les collections Java ;
- utiliser des exceptions personnalisées ;
- organiser le code en packages ;
- le code doit être clair, lisible et commenté si nécessaire.
- **Le code source du projet doit obligatoirement être déposé sur GitHub.**

Le projet doit également contenir une classe de service, par exemple BibliothequeService, qui regroupe la logique principale de l'application.

L'objectif est d'éviter de mettre toute la logique du programme dans la classe Main.

Le projet doit aussi contenir au moins une classe générique, par exemple Gestionnaire<T>.

Cette classe doit permettre de gérer une liste d'objets de différents types, par exemple des documents ou des utilisateurs.

Elle peut contenir des méthodes comme ajouter, supprimer, rechercher ou afficher.

L'objectif est de montrer la compréhension du principe des classes génériques en Java.

---

15. Travail à rendre

Chaque étudiant doit rendre :

1. Le lien du dépôt GitHub contenant le code source Java ;
2. Le diagramme de classes UML ;
3. Le diagramme de cas d'utilisation UML ;
4. Un court rapport au format PDF ou Word.

Le rapport doit contenir :

- une présentation du projet ;
- l'objectif de l'application ;
- le diagramme de classes ;
- le diagramme de cas d'utilisation ;
- une explication des principales classes ;
- une explication des collections utilisées ;
- une explication des exceptions utilisées ;
- des captures d'écran de l'exécution du programme ;
- une conclusion courte.
